Managing Kubernetes secrets

Although you can treat Kubernetes secrets like other configuration, and store them in your version control system, this is not a wise thing to do for several reasons:

- Credentials should be kept secret and distributed to as few people as possible. Storing secrets in a source code repository makes them available to all developers with access to that repository. Although encryption can help, it is easy to get wrong, especially with complex command-line tools such as GPG.
- Secrets should be different in each environment that the service is deployed to; the database password should be different in a development environment compared to your test or production environments. This is the opposite requirement to source code, which should be identical (or close to it) between environments.
- There is almost no value in being able to view the history of secrets. Although you may want to revert the most recent change to a credential if it causes an outage, nobody ever needs to revert to the database password from 2 years ago. If a mistake is made in the encryption of a secret that is hard to change, such as an API key for a third-party service, it's difficult to completely delete the exposed value from a distributed version control system.

A better solution is to either manually manage secrets from the command line, or else use a templating system to generate secrets specific to each environment. Kubernetes supports a templating system called Kustomize, which can generate per-environment secrets based on templates. This allows the template to be checked into version control, but the actual secrets are added during a separate deployment step. See https://kubernetes.io/docs/concepts/configuration/secret/#creating-a-secret-from-generator for more details.

SECURITY OF KUBERNETES SECRETS

Although Kubernetes secrets are easy to use and provide a level of separation between sensitive credentials and other source code and configuration data, they have some drawbacks from a security perspective:

- Secrets are stored inside an internal database in Kubernetes known as etcd. By default, etcd is not encrypted, so anyone who gains access to the data storage can read the values of all